

# Axis conventions

Simon Frost

## Table of contents

|     |                                                                |   |
|-----|----------------------------------------------------------------|---|
| 1   | Overview                                                       | 1 |
| 2   | Setup                                                          | 2 |
| 3   | Row-major with the child fastest: Netica, GeNIe, HUGIN, UAI    | 2 |
| 3.1 | Files written by the tools themselves . . . . .                | 4 |
| 3.2 | from_rowmajor . . . . .                                        | 5 |
| 4   | BIF: rows in any order, or <b>table</b> with the child slowest | 6 |
| 5   | DSC: 0-based indices in parent order                           | 7 |
| 6   | One model, six files                                           | 7 |
| 7   | Summary                                                        | 8 |
| 8   | References                                                     | 8 |

## 1 Overview

Mixing up the layout of conditional probability tables is the most common bug in Bayesian network software. Every file format flattens a table to a list of numbers, and the formats do not agree on the order. This vignette walks through the reference ecological network of SPEC section 45 (Climate  $\rightarrow$  SoilMoisture  $\leftarrow$  Irrigation, SoilMoisture  $\rightarrow$  Vegetation  $\leftarrow$  GrazingPressure, Vegetation  $\rightarrow$  HabitatQuality  $\rightarrow$  Occupancy) in every format and shows how each file's numbers become the same Julia array.

The IR convention (ADR 0002) is: for  $P(Y \mid X_1, \dots, X_k)$  the table has size  $== (n(X_1), \dots, n(X_k), n(Y))$ , parents in the order they are listed in the file, and it is normalised over the last axis.

## 2 Setup

The reference network is a fixture in all six formats, so one `read_network` per format gives six IRs to compare.

```
using BayesianNetworkFormats

files = ["dne/habitat_reference.dne", "xdsl/habitat_reference.xdsl",
"net/habitat_reference.net",
        "bif/habitat_reference.bif", "dsc/habitat_reference.dsc",
        "uai/habitat_reference.uai"]
habitat = Dict{f => read_network(fixture_path(f)) for f in files}
sm = variable(habitat["dne/habitat_reference.dne"], :SoilMoisture)
sm.parents, sm.states, size(sm.table)
```

```
([:Climate, :Irrigation], ["low", "medium", "high"], (3, 2, 3))
```

A helper prints the lines of a fixture that belong to one node:

```
function show_block(name, pattern, n)
    lines = readlines(fixture_path(name))
    i = findfirst(l -> occursin(pattern, l), lines)
    println(join(lines[i:min(i + n - 1, length(lines))], "\n"))
end
```

`show_block` (generic function with 1 method)

## 3 Row-major with the child fastest: Netica, GeNIe, HUGIN, UAI

Netica ([Norsys Software Corp. 2024](#)), GeNIe ([BayesFusion, LLC 2025](#)), HUGIN ([HUGIN EXPERT A/S 2024](#)) and UAI ([UAI Inference Competition 2022](#)) all write the table row-major over (parents..., child): the child's state changes fastest, then the last parent, and the first parent slowest.

The `habitat_reference.*` fixtures used in this section were written by this package, so they show *our rendering* of the convention rather than evidence for it; the row comments are ours too. The evidence is at the end of the section: files written by Netica, GeNIe and a third-party UAI tool that only normalise when read this way.

```
show_block("dne/habitat_reference.dne", "node SoilMoisture", 16)
```

```
node SoilMoisture {
    kind = NATURE;
    discrete = TRUE;
    chance = CHANCE;
    states = (low, medium, high);
    parents = (Climate, Irrigation);
```

```

probs =
  // low    medium  high    // Climate Irrigation
  (((0.7,    0.25,   0.05),    // dry    low
    (0.3,    0.5,    0.2)),    // dry    high
  ((0.3,    0.5,    0.2),    // normal low
    (0.1,    0.4,    0.5)),    // normal high
  ((0.1,    0.4,    0.5),    // wet    low
    (0.05,   0.25,   0.7)));    // wet    high
title = "Soil moisture";
visual V1 {

```

Modern Netica nests one pair of parentheses per parent; old Netica writes the same numbers as one flat list. Both are read by collecting the numbers and ignoring the nesting:

```
show_block("dne/habitat_reference_flat.dne", "node SoilMoisture", 16)
```

```

node SoilMoisture {
  kind = NATURE;
  discrete = TRUE;
  chance = CHANCE;
  states = (low, medium, high);
  parents = (Climate, Irrigation);
  probs =
    // low    medium  high    // Climate Irrigation
    (0.7,    0.25,   0.05,    // dry    low
     0.3,    0.5,    0.2,     // dry    high
     0.3,    0.5,    0.2,     // normal low
     0.1,    0.4,    0.5,     // normal high
     0.1,    0.4,    0.5,     // wet    low
     0.05,   0.25,   0.7);    // wet    high
  title = "Soil moisture";
  visual V1 {

```

The same sequence goes inside GeNIe's <probabilities>:

```
show_block("xds1/habitat_reference.xds1", "<cpt id=\"SoilMoisture\">", 7)
```

```

<cpt id="SoilMoisture">
  <state id="low"/>
  <state id="medium"/>
  <state id="high"/>
  <parents>Climate Irrigation</parents>
  <probabilities>0.7 0.25 0.05 0.3 0.5 0.2 0.3 0.5 0.2 0.1 0.4 0.5 0.1 0.4 0.5
    0.05 0.25 0.7</probabilities>
</cpt>

```

HUGIN nests parentheses like Netica (the % comments are written by this package):

```
show_block("net/habitat_reference.net", "potential (SoilMoisture", 10)
```

```
potential (SoilMoisture | Climate Irrigation)
{
  data =
    ((0.7 0.25 0.05)           % Climate=dry Irrigation=low
     (0.3 0.5 0.2))           % Climate=dry Irrigation=high
    ((0.3 0.5 0.2)           % Climate=normal Irrigation=low
     (0.1 0.4 0.5))           % Climate=normal Irrigation=high
    ((0.1 0.4 0.5)           % Climate=wet Irrigation=low
     (0.05 0.25 0.7)));      % Climate=wet Irrigation=high
}
```

UAI lists the scope of each function with the child last and then the numbers with the last scope variable fastest ([UAI Inference Competition 2022](#)):

```
lines = readlines(fixture_path("uai/habitat_reference.uai"))
println(join(lines[1:12], "\n"))
println("...")
i = findfirst(=="18", lines)
println(join(lines[i:(i + 6)], "\n"))
```

```
BAYES
7
3 2 3 2 3 2 2
7
1 0
1 1
3 0 1 2
1 3
3 2 3 4
2 4 5
2 5 6

...
18
0.7 0.25 0.05
0.3 0.5 0.2
0.3 0.5 0.2
0.1 0.4 0.5
0.1 0.4 0.5
0.05 0.25 0.7
```

### 3.1 Files written by the tools themselves

Two fixtures in this package were written elsewhere. `xdsl/Habitat_Suitability.xdsl` is record 132 of the Bayesian Network Model Archive ([BNMA 2026](#)), written by GeNIe 2.0 ([BayesFusion, LLC 2025](#)); `PreyNumbers` has one three-state row per terrain type, and only the child-fastest reading gives rows that sum to one:

```
show_block("xdsl/Habitat_Suitability.xdsl", "<cpt id=\"PreyNumbers\">", 7)
```

```
<cpt id="PreyNumbers">
  <state id="Low" />
  <state id="Medium" />
  <state id="High" />
  <parents>TerrainTypes</parents>
  <probabilities>0.1 0.3 0.6 0.2 0.6 0.2 0.5 0.45 0.05 0.2 0.4
  0.4</probabilities>
</cpt>
```

```
tiger = read_network(fixture_path("xdsl/Habitat_Suitability.xdsl"))
prey = variable(tiger, :PreyNumbers)
prey.parents, prey.table[1, :], sum(prey.table; dims=2)[: , 1]
```

```
([:TerrainTypes], [0.1, 0.3, 0.6], [1.0, 1.0, 1.0, 1.0])
```

uai/ChestClinic.uai is the *asia* network ([Lauritzen and Spiegelhalter 1988](#)) in UAI BAYES form ([UAI Inference Competition 2022](#)), written by the Merlin library. Read with the last scope variable as the child and the child fastest, it reproduces the published marginal  $P(\text{dysp} = \text{yes}) = 0.436$ , a number this package had no hand in:

```
chest = read_network(fixture_path("uai/ChestClinic.uai"))
variable(chest, :X7).parents, round.(marginal(chest, :X7); digits=5)
```

```
([:X1, :X5], [0.43597, 0.56403])
```

Both files are recorded with their origin and licence in `test/fixtures/LICENSES.md`. The Netica-written BNMA models ([BNMA 2026](#); [Kotalik et al. 2026](#)) in `EcologicalBayesianNetworks.jl/models/` play the same role for `.dne`: the test suite reads them, with their `#k` findings and levels ([Norsys Software Corp. 2024](#)), whenever the sibling checkout is present.

### 3.2 from\_rowmajor

All four are converted with one function. Given the flat list and the dimensions (`n(p1)`, ..., `n(pk)`, `n(child)`), `from_rowmajor` reshapes with the dimensions reversed (so that Julia's column-major order matches the file's row-major order) and then reverses the axes back:

```
vals = [0.7, 0.25, 0.05, 0.3, 0.5, 0.2, 0.3, 0.5, 0.2, 0.1, 0.4, 0.5, 0.1, 0.4, 0.5,
0.05, 0.25, 0.7]
t = from_rowmajor(vals, (3, 2, 3))
t == sm.table
```

```
true
```

```
t[1, 2, :] # Climate = dry, Irrigation = high
```

```
3-element Vector{Float64}:  
 0.3  
 0.5  
 0.2
```

A plain reshape would silently give a different table whose rows do not sum to one:

```
wrong = reshape(vals, 3, 2, 3)  
wrong[1, 2, :], sum(wrong[1, 2, :])
```

```
([0.3, 0.1, 0.05], 0.45)
```

to\_rowmajor is the inverse and is what every writer uses:

```
to_rowmajor(sm.table) == vals
```

```
true
```

## 4 BIF: rows in any order, or **table** with the child slowest

The bnlearn dialect of BIF ([Cozman 1998](#); [Scutari 2010](#)) gives one row per parent configuration, labelled with the parent states in parent order. Rows may appear in any order, so the reader indexes them by state name rather than position (bnlearn happens to write the first parent fastest):

```
show_block("bif/habitat_reference.bif", "probability ( SoilMoisture", 8)
```

```
probability ( SoilMoisture | Climate, Irrigation ) {  
  (dry, low) 0.7, 0.25, 0.05;  
  (normal, low) 0.3, 0.5, 0.2;  
  (wet, low) 0.1, 0.4, 0.5;  
  (dry, high) 0.3, 0.5, 0.2;  
  (normal, high) 0.1, 0.4, 0.5;  
  (wet, high) 0.05, 0.25, 0.7;  
}
```

pgmpy and the original BIF specification ([Cozman 1998](#)) use a table line instead. Its numbers are in *header* order, (child, parent1, ..., parentk), with the last variable fastest, so the child is now the slowest index. The reader reshapes with from\_rowmajor over (n(child), n(p1), ..., n(pk)) and then moves the child axis to the end:

```
show_block("bif/sprinkler_table.bif", "probability ( Grass", 3)
```

```
probability ( Grass | Sprinkler, Rain ) {
  table 0.99, 0.9, 0.8, 0.0, 0.01, 0.1, 0.2, 1.0 ;
}
```

```
grass = variable(read_network(fixture_path("bif/sprinkler_table.bif")), :Grass)
grass.parents, grass.table[1, 1, :], grass.table[2, 2, :] # (on, yes) and (off,
no)
```

```
([:Sprinkler, :Rain], [0.99, 0.01], [0.0, 1.0])
```

## 5 DSC: 0-based indices in parent order

DSC rows ([Kadie et al. 2001](#)) carry 0-based state indices instead of names, again in parent order and in any order:

```
show_block("dsc/habitat_reference.dsc", "probability ( SoilMoisture", 8)
```

```
probability ( SoilMoisture | Climate, Irrigation ) {
  (0, 0) : 0.7, 0.25, 0.05;
  (1, 0) : 0.3, 0.5, 0.2;
  (2, 0) : 0.1, 0.4, 0.5;
  (0, 1) : 0.3, 0.5, 0.2;
  (1, 1) : 0.1, 0.4, 0.5;
  (2, 1) : 0.05, 0.25, 0.7;
}
```

## 6 One model, six files

After conversion the six IRs agree entry for entry, and the brute-force joint distribution gives the same marginals from each:

```
ref = habitat["dne/habitat_reference.dne"]
for f in files
  same = isequent(habitat[f], ref; titles=false, positions=false,
comments=false, extras=false)
  println(rpad(f, 32), same ? "equivalent" : "DIFFERENT",
round.(marginal(habitat[f], :Occupancy); digits=6))
end
```

|                             |            |                      |
|-----------------------------|------------|----------------------|
| dne/habitat_reference.dne   | equivalent | [0.523759, 0.476241] |
| xdsl/habitat_reference.xdsl | equivalent | [0.523759, 0.476241] |
| net/habitat_reference.net   | equivalent | [0.523759, 0.476241] |
| bif/habitat_reference.bif   | equivalent | [0.523759, 0.476241] |
| dsc/habitat_reference.dsc   | equivalent | [0.523759, 0.476241] |
| uai/habitat_reference.uai   | equivalent | [0.523759, 0.476241] |

The direction of the check matters: SoilMoisture has parents of different sizes (Climate has three states, Irrigation two), so swapping the parent axes changes the shape and is caught by `validate`. Two parents with the same number of states would not be caught by shape alone; the cross-format equality tests and the brute-force marginal of *asia* ( $P(\text{dysp} = \text{yes}) = 0.4360$ ) (Lauritzen and Spiegelhalter 1988) in the test suite pin the convention for that case.

## 7 Summary

Netica, GeNIe, HUGIN and UAI flatten a table row-major over (parents..., child) with the child fastest, so `from_rowmajor` and `to_rowmajor` are the only conversion the package needs; BIF table lines put the child slowest, and BIF and DSC rows are labelled rather than positional. Tool-written files pin the convention independently: only the child-fastest reading normalises `Habitat_Suitability.xdsl`, and only it reproduces the published *asia* marginal from `ChestClinic.uai`. The next vignette, *Influence diagram files*, adds decision and utility nodes, whose tables follow the same convention over their parents.

## 8 References

- BayesFusion, LLC. 2025. *GeNIe Modeler and SMILE Engine Documentation*. <https://support.bayesfusion.com/docs/>.
- BNMA. 2026. *The Bayesian Network Model Archive*. <https://bnma.co/bnrepo/>.
- Cozman, Fabio Gagliardi. 1998. *The Interchange Format for Bayesian Networks, Version 0.15*. Carnegie Mellon University. <http://www.cs.cmu.edu/~fgcozman/Research/InterchangeFormat/>.
- HUGIN EXPERT A/S. 2024. *HUGIN API Reference Manual: The NET Language*. <https://download.hugin.com/webdocs/manuals/api-manual.pdf>.
- Kadie, Carl M., David Hovel, and Eric Horvitz. 2001. *MSBNx: A Component-Centric Toolkit for Modeling and Inference with Bayesian Networks*. MSR-TR-2001-67. Microsoft Research.
- Kotalik, Christopher J., Freya E. Rowland, Bruce G. Marcot, et al. 2026. “Causal Networks to Inform Decisions for Ecological Restoration.” *Environmental Management* 76 (7). <https://doi.org/10.59381/ofwegrfqyo>.
- Lauritzen, Steffen L., and David J. Spiegelhalter. 1988. “Local Computations with Probabilities on Graphical Structures and Their Application to Expert Systems.” *Journal of the Royal Statistical Society, Series B* 50 (2): 157–224. <https://doi.org/10.1111/j.2517-6161.1988.tb01721.x>.
- Norsys Software Corp. 2024. *Netica File Formats*. [https://www.norsys.com/WebHelp/NETICA/X\\_File\\_Formats.htm](https://www.norsys.com/WebHelp/NETICA/X_File_Formats.htm).

Scutari, Marco. 2010. “Learning Bayesian Networks with the bnlearn R Package.” *Journal of Statistical Software* 35 (3): 1–22. <https://doi.org/10.18637/jss.v035.i03>.

UAI Inference Competition. 2022. *Model File Format*. <https://uaicompetition.github.io/uci-2022/file-formats/model-format/>.